CUSTOM SUBAGENTS UNDER THE HOOD

Two required fields. Eleven optional ones. The gap between a useful subagent and expensive context lives entirely in the optional eleven.

- Frontmatter anatomy
- Tool & permission control
- Scope & priority order
- Persistent agent memory
- When to delegate


Pulkit Tyagi
Senior Gen-AI Engineer

WHAT THEY ACTUALLY DO

> *A subagent does verbose work in its own context window — and returns only the summary.*
>
> — The whole design, in one line

Every other benefit — cost control, tool restrictions, model swaps — follows from that single fact: the noise stays in the subagent. Your main conversation only sees the answer.


Pulkit Tyagi
Senior Gen-AI Engineer

THE FILE IS JUST MARKDOWN

YAML frontmatter on top, system prompt below. **Only `name` and `description` are required** — everything else has a default.

`.claude/agents/code-reviewer.md`

```
---
name: code-reviewer
description: Reviews code for quality
tools: Read, Glob, Grep
model: sonnet
---

You are a code reviewer. Analyze
the diff and provide specific,
actionable feedback on quality,
security, and best practices.
```


Pulkit Tyagi
Senior Gen-AI Engineer

FIELDS THAT ACTUALLY MATTER

Two required. The rest are levers — pick the ones that match the job, ignore the rest.

FIELD	WHAT IT DECIDES
name	Identifier (lowercase, hyphens)
description	When Claude delegates
tools	Allowlist of tools
disallowedTools	Subtract from inherited
model	haiku / sonnet / opus / inherit
memory	Cross-session knowledge
permissionMode	How prompts are handled

Pulkit Tyagi
Senior Gen-AI Engineer

TOOL ACCESS
IS THE SECURITY KNOB

Skip both fields and the subagent inherits everything — including every MCP tool you didn't mean to share.

TOOLS (ALLOWLIST)	DISALLOWEDTOOLS
Specifies the exact set	Subtracts from inherited
Use for read-only researchers	Use when you mostly trust it
`Read, Grep, Glob, Bash`	`Write, Edit`
No MCP unless listed	Keeps Bash + MCP available


Pulkit Tyagi
Senior Gen-AI Engineer

SCOPE LADDER
HIGHEST WINS

Five places to drop a subagent file. Same name in two places? The higher-priority scope wins.

- → **Managed settings** — org-wide, deployed by admins
- → **--agents CLI flag** — current session only, never saved
- → **.claude/agents/** — project, commit to git
- → **~/.claude/agents/** — all your projects
- → **Plugin agents/** — wherever the plugin is enabled


Pulkit Tyagi
Senior Gen-AI Engineer

MEMORY MAKES IT A SPECIALIST

Set **memory: project** and the agent gets a directory it reads and writes between conversations. Patterns compound.

`.claude/agents/db-reader.md`

```
---
name: db-reader
description: Run read-only SQL.
             Use proactively for
             data questions.
tools: Bash, Read
model: haiku
memory: project
---

Cheap, fast, persistent.
First 200 lines of MEMORY.md
auto-load on every run.
```

Pulkit Tyagi
Senior Gen-AI Engineer

REACH FOR ONE WHEN...

Subagents start fresh and lose to the main conversation on latency. Use them when the task fits this shape:

- → **Verbose output** — test logs, search results you won't reference
- → **Self-contained** — returnable as a clean summary
- → **Strict tool boundary** — read-only, sandboxed, restricted
- → **Parallel research** — auth, DB, API explored at once
- → **Cost-sensitive** — cheap model for high-volume side work

Pulkit Tyagi

SENIOR GEN-AI ENGINEER

VERBOSE WORK GOES IN. SUMMARY COMES OUT.

That's the whole design. Your main thread stays clean. Tools stay scoped. Costs stay routed to **Haiku** when they should. And memory turns a one-shot script into a specialist that compounds.

Which task in your loop is verbose enough to deserve its own subagent?

LIKE COMMENT SAVE REPOST